Scaling a Flash Sale Service

Ella Bredow, Felix Hauptmann, Efe Sonugür | Prototyping Presentation | SE | July 13, 2026

Project Description

SE3S Prototype

Assignment

01 **Project**

- Ticket Flash Sale Prototype

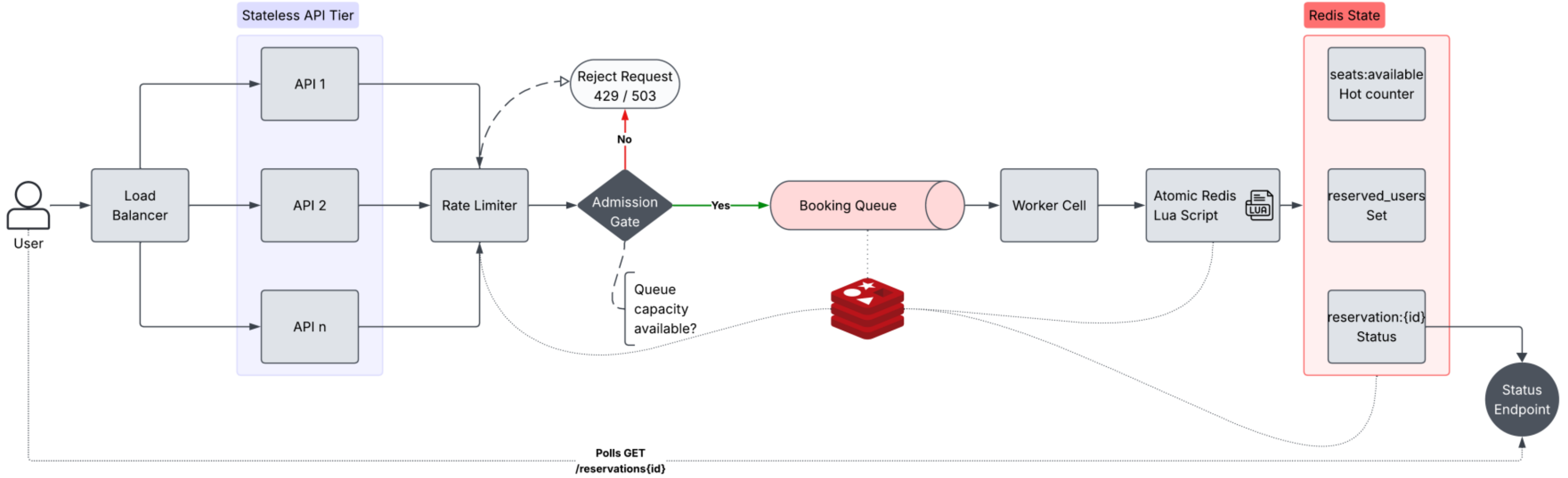
02 **Goal**

- Development of a scalable ticket flash sale application with GCP deployment

03 **Solution**

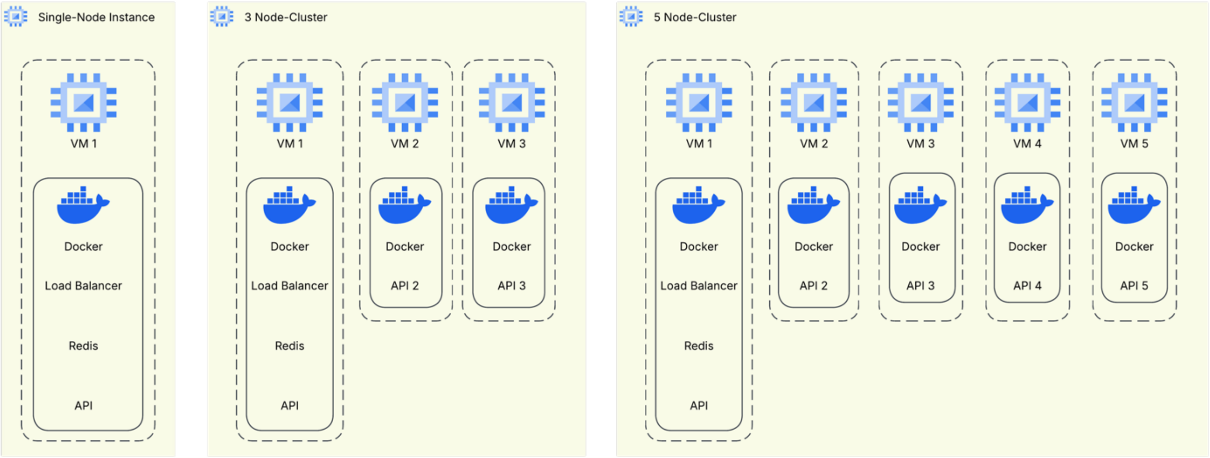
- Provides a scalable solution for processing high volumes of simultaneous ticket requests

Application Architecture



Cluster Setup

- Scale FastAPI because Worker handle shared state
- Better scaling results by FastAPI
    - FastAPI handle short-lived request
    - Workers have longer jobs



Used Patterns

Constant Work

Workers:
- Processes a fixed number of queue items per interval

Bounded Queue

Admission Gate:
- Checks queue length & rejects if queue is full

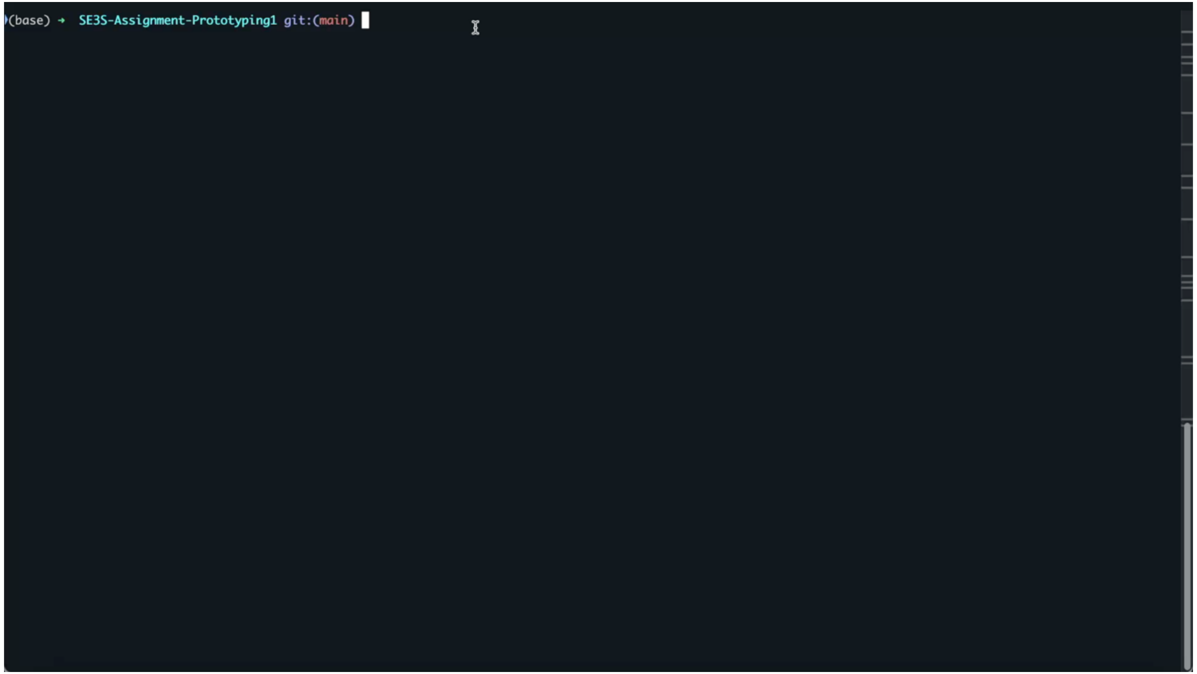
Load Balancer

NGINX:
- Distributes requests using least connections

Token Bucket

Redis Rate Limiter:
- Limits repeated booking attempts per use

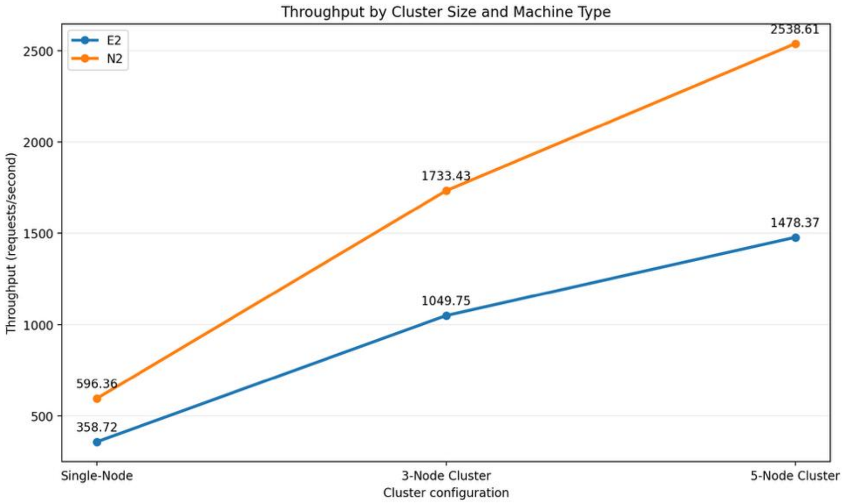
Demo



Scalability Results – Baseline Test

Cluster	Req/s – E2	Req/s – N2
Single-Node Instance	358.72	596.36
3 Node-Cluster	1049.75	1733.43
5 Node-Cluster	1478.37	2538.61

→ Almost Linear!



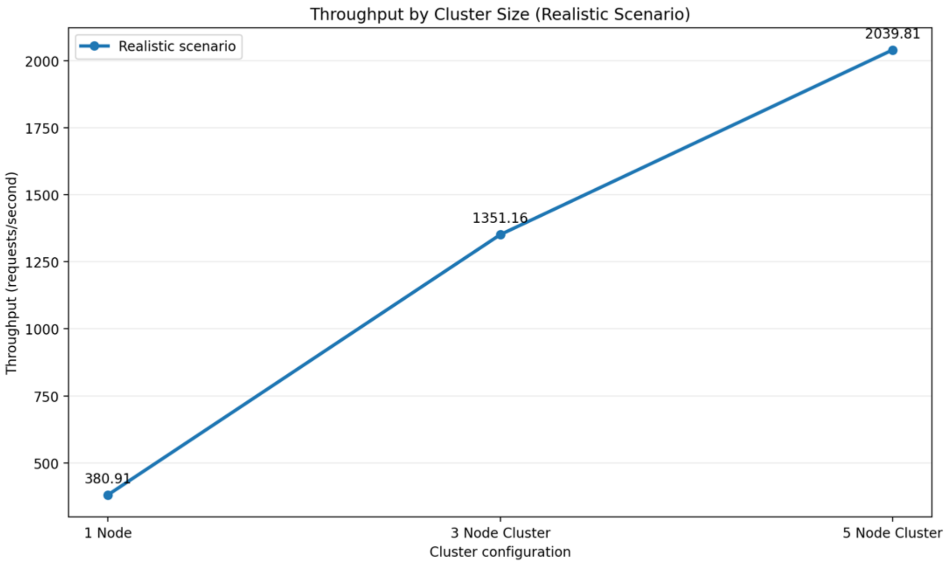
Generated using ChatGPT

 Ella Bredow, Felix Hauptmann, Efe Sonugür | SE3S | Group 1

Scalability Results – Realistic Scenario Test

Cluster	Req/s – E2
Single-Node Instance	380.91
3 Node-Cluster	1351.16
5 Node-Cluster	2039.81

→ Confirms performance in a realistic scenario



Generated using ChatGPT

Limitations & Possible Mitigation

Possible Bottlenecks:

- **Load Balancer**
  - Deploy multiple instances
- **Admission Gate**
  - Scaling requires more communication
- **Worker**
  - Possibility to also scale horizontally
- **Lua Script**
  - Mitigate by sharding

Optimize the Replica:

- Move high impact service to separate VM

Any questions?